Composite keys

A composite key is when you use **multiple columns together** as the primary identifier.

```
// Single Primary Key
model User {
  id    String @id @default(uuid())  // Just one column as primary key
  email String @unique
}

// Composite Primary Key: Here it makes sense to use Composite keys
// Since a user cannot be in two conversation simultaneuously.
model ConversationParticipant {
  conversationId String
  userId         String

  @@id([conversationId, userId])  // Two columns together = primary key
}
```

Can Also use Composite Keys here

```
model UserPresence {
  userId    String
  date      DateTime
  isOnline  Boolean

  @@id([userId, date])  // One presence record per user per day
}

// OR You Can Also Do it here
model Comment {
  postId    String
  commentId Int
  text      String
```

```
  @@id([postId, commentId])  // Comment #1 on Post A, Comment #1 on Post B
}
```

Here don't use Composite keys

```
// Yaha sense nhi banti ek hi id se krenge identify user ko unique...
model User {
  id    String @id @default(uuid())  // User id would identify user
  email String @unique
}

model Message {
  id      String @id @default(uuid())  // Unique id for every message
  text    String
  sentAt  DateTime
}
```

Example of composite keys

Create a `Student` and `Course` and `Enrollment` models

▼ Student and Course

```
model Student {
  id          String       @id @default(uuid())
  name        String
  enrollments Enrollment[]
}

model Course {
  id          String       @id @default(uuid())
  name        String
  enrollments Enrollment[]
}
```

▼ Enrollment

```
model Enrollment {
  studentId  String
  courseId   String
  enrolledAt DateTime @default(now())
  grade      String?

  student Student @relation(fields: [studentId], references: [id])
  course  Course  @relation(fields: [courseId], references: [id])

  @@id([studentId, courseId])  // COMPOSITE PRIMARY KEY
}
```

▼ Insert into Student and Course

```
// Create students
const alice = await prisma.student.create({
  data: { name: "Alice" }
});

const bob = await prisma.student.create({
  data: { name: "Bob" }
});

// Create courses
const math = await prisma.course.create({
  data: { name: "Mathematics" }
});

const physics = await prisma.course.create({
  data: { name: "Physics" }
});
```

▼ Enroll Students To Courses

```
// Alice enrolls in Math
await prisma.enrollment.create({
  data: {
    studentId: alice.id,
    courseId: math.id,
    grade: "A"
  }
});

// Alice enrolls in Physics
await prisma.enrollment.create({
  data: {
    studentId: alice.id,
    courseId: physics.id,
    grade: "B+"
  }
});

// Bob enrolls in Math
await prisma.enrollment.create({
  data: {
    studentId: bob.id,
    courseId: math.id,
    grade: "A-"
  }
});

// Would look something like this
studentId  | courseId | grade
 alice-id  | math-id  | A
 alice-id  | phys-id  | B+
 bob-id    | math-id  | A-
```

▼ Find all Enrollments of a Student

```
// Find all of Alice enrollments
const aliceEnrollments = await prisma.enrollment.findMany({
  where: {
    studentId: alice.id
  },
  include: {
    course: true  // Include course details
  }
});
```

```
//
// [
//   { studentId: "alice-id", courseId: "math-id", grade: "A", course: { name: "Mather
//   { studentId: "alice-id", courseId: "phys-id", grade: "B+", course: { name: "Phys:
// ]
```

▼ Find all Course of a Student

```
// Find all students enrolled in Math
const mathStudents = await prisma.enrollment.findMany({
  where: {
    courseId: math.id
  },
  include: {
    student: true  // Include student details
  }
});

//
// [
//   { studentId: "alice-id", courseId: "math-id", grade: "A", student: { name: "Alic
//   { studentId: "bob-id", courseId: "math-id", grade: "A-", student: { name: "Bob"
// ]
```

▼ Updating Student data

```
// Update Alice Math grade from A to A+
await prisma.enrollment.update({
  where: {
    studentId_courseId: {  // Composite key to identify
      studentId: alice.id,
      courseId: math.id
    }
  },
  data: {
    grade: "A+"
  }
});
```

▼ Working of Composite Keys

```
@@id([field1, field2, field3])
where: {
  field1_field2_field3: {  // Join fields with underscores
    field1: value1,
    field2: value2,
    field3: value3
  }
}
```

Thank you!! Enjoyyy